

李乐岩

3073751449@qq.com | +86 18805506898 | GitHub: <https://github.com/leyancode>

爱丁堡大学 University of Edinburgh 英国 | 2025.09 – 2026.12

高性能计算与数据科学 硕士

课程与毕业论文 2026.08 完成 | 可入职时间 2026.09.01 | 学位证书 2026.12 获得

利物浦大学 University of Liverpool 英国 | 2023.09 – 2025.06

计算机科学 本科, 西交利物浦大学 2+2 项目, 一等荣誉学位

技术能力

编程与工程: Python、C/C++、SQL; Linux、Bash、Git、pytest, 了解 Docker; 具备用自动化脚本管理实验并构建可复现流程的经验。

AI Agent 与 LLM 应用: 了解 Agent 常见架构、prompt engineering、记忆与多轮对话状态管理; 了解向量检索路线 (文本切分、embedding、FAISS / Chroma、top-k 语义检索) 及其与词法检索在可解释性与同义词召回上的取舍。

后端与数据库: 了解 REST / HTTP、限流、幂等性与可观测性; SQLite / MySQL 的索引、事务与查询优化; 了解 Redis 缓存场景与 TTL、穿透 / 击穿。

机器学习与高性能计算: 熟悉 PyTorch 与 Pandas / NumPy / Matplotlib; 理解 MPI 进程通信、OpenMP 线程并行与 hybrid 模式, 以及 strong scaling、parallel efficiency、cache locality 与 NUMA。

项目经历:

面向高性能计算 Benchmark 的 Agent 式性能分析系统 github.com/leyancode/hpc-benchmark-agent

项目描述: 基于英国国家超算 ARCHER2 上真实的 PETSc benchmark 数据, 构建“确定性工具 + 可评测路由”的 Agent 式性能分析系统: 结果结构化为 SQLite, 分析任务封装为确定性工具, 中文规则 router 作为可验证 baseline; 接入 LLM function calling router 后用自建 30 问五类中文评测集对两者打分——规则 83.3%、LLM 100%。数据为 ARCHER2 上 PETSc scaling 基准的真实运行导出 (5 个 2D 规模、1M-40.96M unknowns)。

技术栈: Python、SQLite、pytest、FastAPI、OpenAI SDK、PETSc、MPI、OpenMP、Slurm、Linux

- 分层架构与同构契约: 数据导入 / 指标 / 工具 / 检索 / 路由 / 可观测层, 加 CLI 与 FastAPI 两个平级且互不依赖的进程边界; 规则 router 与 LLM tool call 输出严格同构 (intent + 参数), 故可共用同一套评测集、切换只需改一行 dispatch; 各层合同由 229 项自动化测试钉死, 全量离线可跑。
- 中文规则 router: 关键词意图分类 + 问题规模解析 (阿拉伯数字、「100 万」混写、中文数字「一百万」), 无单位纯数字须 ≥ 4 位以排除「128 核」被误读为规模; 缺参数主动追问、超范围明确拒答。
- 确定性工具与字节级报告: 最优配置查询、MPI-only 与 hybrid 模式对比 (数据只含 ranks / threads, 故用保守标签 mpi_like / hybrid_like)、扩展性报告; 同一数据库生成的 Markdown 报告字节级一致 (刻意不含时间戳), 使「文档是否漂移」退化为一次 git diff。
- LLM function calling 与防御式解析: tool schema 的 description 写明工具边界, unknowns 刻意可选——省略参数即「追问」、不调用工具即「拒答」; 未调用工具、幻觉工具名、arguments 非合法 JSON 三类异常统一收敛为显式 unknown, 绝不猜参数以避免「静默算错」; API 失败向上抛出而非回退规则 router, 以免污染评测 (测试经 FakeLLM 注入离线运行)。
- 评测机制与消融结论: 自建 30 问五类中文评测集 (正常 / 追问 / 拒答 / 数字陷阱 / 规则已知会错的对抗题), 期望值为 tool call 而非答案文本, harness 对 router 实现无知; 消融显示同一模型不加 system prompt 仅 63.3%——准确率主要由工具说明书与策略 prompt 决定; 报告如实标注局限 (题量 30、单模型单次)。
- 工程化外围: trace_id 贯穿的 JSON Lines 事件流可完整回放一次执行, 脱敏收敛在单一出口点; FastAPI 暴露 POST /ask, 缺 key 映射 503 而非 500, 与 CLI 的退出码 2 各自独立; 最小 RAG 用 IDF 加权词法检索 (纯标准库、刻意不用向量库), 检索不到即返回空、不注入, 未开启时 messages 与评测时逐字节一致以保结论可比。

多智能体 Actor-Critic 方法中的价值函数分解研究 (本科毕业设计) github.com/leyancode/marl-value-factorisation

项目描述: 在 MAPPO 的 critic 上分别接入 VDN 加性分解与 QMIX 单调 mixing 做单变量对照: 三个变体共用同一套 runner / buffer / shared RNN actor 与全部 PPO 超参, 差异只在 critic 分支。结论与直觉相反——表达能力更强的 critic 没有赢: 稀疏奖励的 Predator-Prey (300k steps) 上, 最简单的加性分解唯一学到正回报 (≈ 150), 单体 critic 始终为负 ($-300 \rightarrow -30$), 单调 mixing 的 critic loss 起手即达 5.4×10^4 、200k 步后二次发散。代码与实验图表开源:

技术栈: Python、PyTorch、PyMARL、MAPPO、VDN、QMIX、CTDE、PPO、PettingZoo / MPE2

- 根因诊断: MAPPO-QMIX 的 TD 目标沿用 value-based QMIX 的 $\max_a Q_i$ (贪婪动作), 而 actor 是 on-policy, 相减得到的 td_error 并非合法 advantage 而是系统性偏乐观的量, 可解释「回报冲过理论最优值后崩塌」; 另定位 hypernet 经 softplus 生成的 mixing 权重 ($\text{softplus}(0) \approx 0.69$) 对局部 Q 的梯度做乘性放大, 是 critic loss 起手即达 5×10^4 的直接原因。
- 负结果与自我批判: 去掉单调约束的 non-monotonic mixer 持续梯度爆炸、未在预算内收敛, 如实写入论文; 进一步指出三个 learner 实际都只产出一个共享的标量 advantage, 故分解的收益来自表示与梯度路径, 而非真正的 per-agent credit assignment; 2026 年整理为可复现仓库 (修正合作边界、锁定 CPU 依赖、补 pytest 与 CI)。

